PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Contents

1. Project Overview 3
2. Background & Business Objective 3-4
3. Product Purpose 4
4. System Functionalities 5-6
5. User Stories & Use Cases 7
6. Features Included (Scope Definition) 8
7. Features Not Included (Scope Control) 8
8. Assumptions & Constraints 8
9. Risks & Questions Throughout Development 8

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: Last Minute ChatRojak

Version: 1.0

1. Project Overview

- **Problem Statement:** Academic and collaborative communication is heavily reliant on messaging platforms like WhatsApp, leading to information overload. Critical data—such as deadlines, task assignments, and club announcements—frequently gets buried within casual chat history. This forces students into a manual, error-prone tracking process that increases stress and causes missed deadlines.
- **Target Domain:** Optimizing academic productivity and task management for students and collaborative campus organizations.
- **Proposed Solution Summary:** A streamlined integration or platform that automatically extracts and categorizes actionable tasks from chat .txt file. By converting conversational noise into structured to-do lists and calendar events, the tool shifts the student's role from a manual archivist to an organized executor, ensuring no deadline is overlooked.

2. Background & Business Objective

- **Background of the Problem:** For many years, professionals and students have relied on chat applications like WhatsApp for team communication. This results in critical tasks, deadlines, and decisions becoming buried within thousands of lines of unstructured, conversational text. The process of extracting and organizing this information is entirely manual, requiring individuals to painstakingly read through chat logs and copy-paste action items into separate to-do lists or calendars. This process is slow, prone to human error, and creates a major disconnect between where work is discussed and where it is tracked.

© UMHackathon 2026
Email: umhackathon@um.edu.my

- **Importance of Solving This Issue:** Accelerating the time to convert a conversation into an actionable plan from hours to seconds. By automating the extraction and prioritization of tasks, the industry can ensure that no commitment is missed and that team members can focus on execution rather than administration. Breaking the barrier of manual organization, anyone can get a clear, prioritized overview of their commitments without needing to constantly re-read chat histories
- **Strategic Fit / Impact:** It aligns with Google's AI-Native Development Lifecycle Ecosystem. The project's flexible architecture, which integrates flagship models like Google's Gemini and Z Ai's GLM, uses AI as the core engine for its 3-step reasoning pipeline (Extract, Plan, Validate). By connecting to external services like Google Calendar, it facilitates the developer's and user's entire workspace, making existing tools smarter.

3. Product Purpose

3.1. Main Goal of the System: To provide an AI-powered assistant that automatically converts unstructured chat conversations into an organized and prioritized action plan.

3.2. Intended Users (Target Audience):

- Businessmen & Project Managers
- Club Organizers & Event Planners
- Students collaborating on group projects
- Small Business Owners
- Freelancers and small, agile teams
- Anyone who feels overwhelmed by tasks and decisions buried in their chat logs.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. System Functionalities

4.1. Description: The system operates as an AI-powered productivity engine that transforms unstructured natural language from chat logs into a structured and prioritized action plan. Using a multi-step reasoning pipeline, the intelligence captures tasks, deadlines, dependencies, and automatically categorizes them to provide instant clarity.

4.2. Key Functionalities:

- Vibe Designing: Translates raw, conversational chat logs into a clean, structured list of actionable tasks, eliminating the need for manual copy-pasting and note-taking.
- Conversational Editing: Facilitates automatic decision-making by using an AI to analyze each task's urgency and importance, sorting them into an Eisenhower Matrix (Do Now, Plan, etc.) on the main dashboard.
- Versioning & Infinite Canvas: The AI can deconstruct large, complex goals into smaller, manageable sub-tasks and allows users to establish dependencies between items to create a clear project timeline.
- Export- Component Based: Enables connection of the extracted tasks to the user's existing workflow by syncing deadlines to Google Calendar and sending notifications.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: LLMs Used: The system is architected to be model-agnostic, primarily using Z AI GLM/Google's Gemini as its reasoning engines. The active model is configurable via the .env file. Justification: Both models are excellent at following complex instructions and reliably generating the structured JSON output that our system requires to function.

© UMHackathon 2026
Email: umhackathon@um.edu.my

4.3.2. Prompting Strategy: We use multi-step agentic prompting. We break down the complex problem into three simple, specialized AI agent calls (Extract, Plan, Validate). This approach is more accurate, reliable, and easier to maintain than using a single, large prompt.

4.3.3. Context & Input Handling: The system is fundamentally designed to handle unstructured input; it is our core use case. The AI Extractor agent is specifically prompted to find tasks, deadlines, and context within messy, conversational chat logs. The maximum input size is 500kb, which is configured in the Express server (server.js). We do not use truncation or chunking. If a user attempts to upload a file larger than 500kb, the server immediately rejects the request with a 413 Payload Too Large error, and the user is shown an error message in the UI.

4.3.4. Fallback & Failure Behavior:

- Automatic Retry: For transient network issues or temporary API failures, the withRetry function in src/client.js automatically retries the API call up to three times before failing.
- Graceful Error State: If the model returns a hallucinated or malformed response (e.g., plain text instead of JSON), the extractJson function's try/catch block prevents a crash. It safely returns an empty object, allowing the system to continue running without interruption.
- Human Escalation Path: When the main promptChain receives this empty object, it recognizes the AI has failed. Instead of silently ignoring it, it triggers a human escalation path: the original text is saved as a "draft" task and flagged in the UI for the user to manually review and correct.

© UMHackathon 2026

5. User Stories & Use Cases

- **User Stories:** *Example:* "As a busy project manager, I want to upload my team's WhatsApp chat log so that the system automatically creates a prioritized to-do list for me, without me having to read through everything manually."
- **Use Cases (Main Interactions):**

  Prompt to UI Flow: A user uploads a .txt chat file. The system's AI pipeline then parses the entire conversation, identifies actionable tasks and deadlines, automatically categorizes them by urgency and importance, and displays the final, structured tasks in the Power Grid UI.

  Flow Generation: A user identifies a complex task like "Launch new marketing campaign." The Planner AI suggests a list of sub-tasks (e.g., "Draft ad copy," "Design visuals," "Configure ad spend"). The user can then add dependencies between these sub-tasks to create a clear project plan.

6. Features Included (Scope Definition)

- An AI that reads our messy group chat files and automatically finds all the assignments and deadlines.
- A dashboard that automatically sorts our tasks into "Do Now," "Plan," and "Later" so we know what to work on first.
- One-way syncing to our Google Calendars so deadlines show up automatically.
- Support for uploading .txt files exported from WhatsApp.

© UMHackathon 2026

Email: umhackathon@um.edu.my

7. Features Not Included (Scope Control)

- The app doesn't connect directly to WhatsApp; we have to upload the export chat file manually.
- It doesn't support other chat apps like Telegram or Discord, only WhatsApp .txt files.
- It only processes the chat once; it can't update if there are new messages.

8. Assumptions & Constraints

- **LLM Cost Constraint:** We assume the token cost per chat file will be manageable. Our design helps by stopping the AI process early if the first step (the Extractor) sees that a message isn't a real task. This saves us from wasting money on the other three AI calls.
- **Technical Constraints:** The app is a simple web page (HTML/CSS). It's not a native mobile app for iOS or Android. It also only works with .txt files from WhatsApp, not other chat apps.
- **Performance Constraints:** Processing a big chat file can be slow because it has to make multiple calls to the AI. We put a 500kb file size limit to stop it from taking forever. The user will see a loading spinner while they wait.
- **User Input:** The app can't do anything without a user uploading a chat file. Also, if our group chat is super messy with lots of slang and typos, the AI might get confused and make mistakes.

9. Risks & Questions Throughout Development

- AI Hallucination: How are we ensuring the AI isn't just making up tasks or deadlines that were never actually mentioned in the chat?
- Data Accuracy: Do we need to double-check every single task the AI makes to make sure the dates and details are correct, or can we trust it?
- Language Interpretation: What will happen if the AI can't understand our group's slang, inside jokes, or all the typos in our chat?

© UMHackathon 2026

Email: umhackathon@um.edu.my